

AI Assisted Coding

Name : B.Thanmai

H.no : 2303A52126

Batch : 33

Course Code : 23CS002PC304

Assignment Number : 15.5

Lab 15 – Backend API Development: Creating RESTful services with AI

Task 1 – Student Attendance API Task:

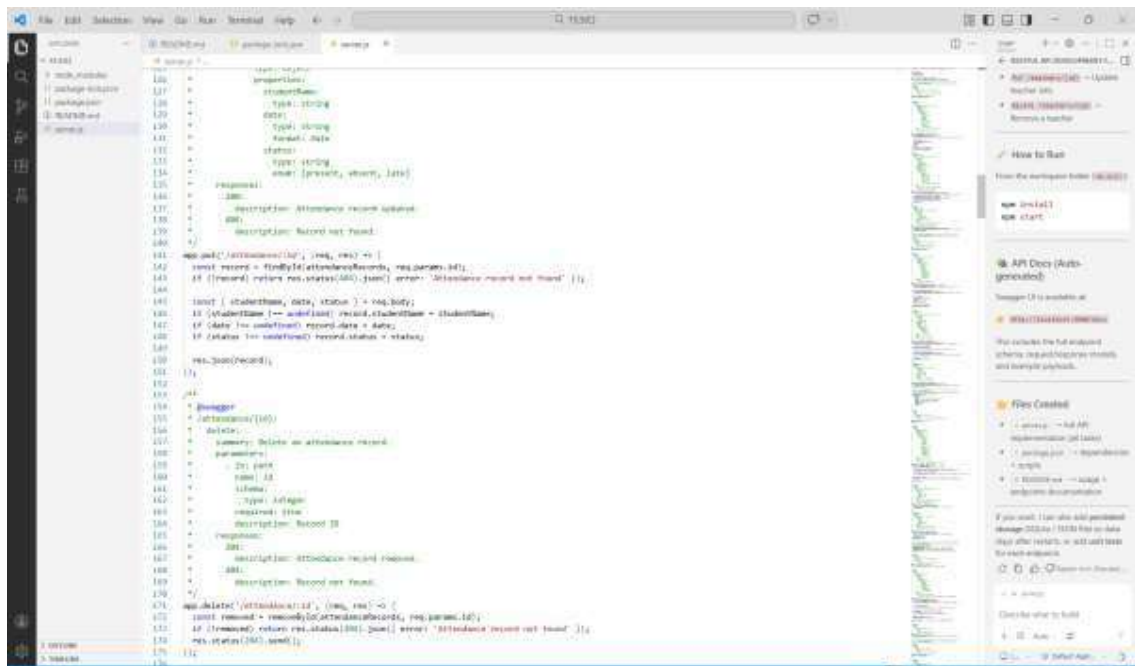
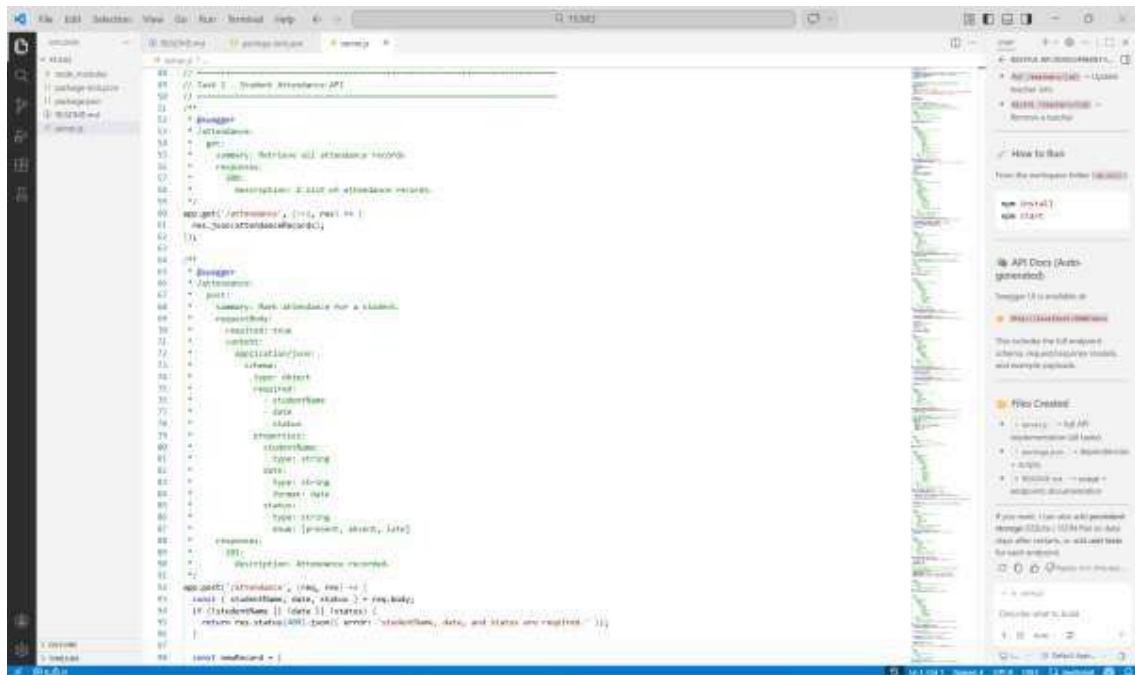
Create a RESTful API for student attendance tracking.

Instructions:

- Endpoints required: o GET /attendance → View attendance records o POST /attendance → Mark attendance o PUT /attendance/{id} → Update attendance record o DELETE /attendance/{id} → Remove attendance entry
- Ensure JSON-based request and response handling.

Expected Output:

- API for maintaining and updating attendance records.

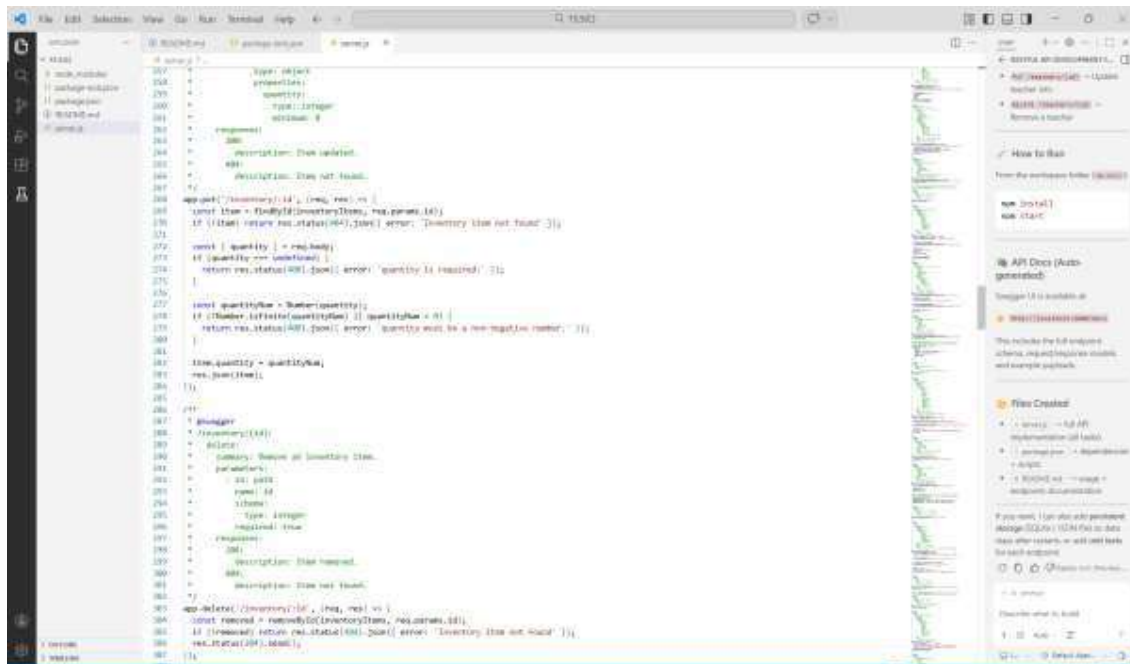


Task 2 – Inventory Management API Task:

Develop a REST API for warehouse inventory management.

Instructions:

- Endpoints required:



Task 3 – Hotel Room Booking API Task:

Develop a RESTful API for hotel room booking.

Instructions:

- Endpoints required:

o GET /rooms → View available rooms o

POST /rooms/book → Book a room o GET

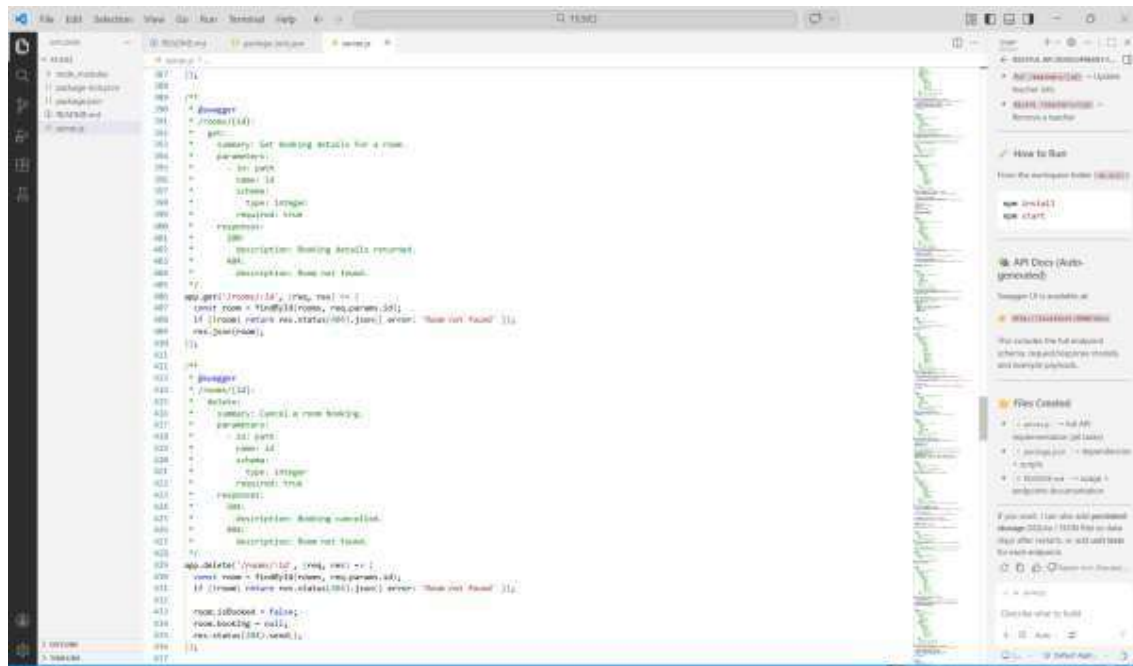
/rooms/{id} → View booking details o

DELETE /rooms/{id} → Cancel booking

- Implement error handling for unavailable rooms.

Expected Output:

- API simulating hotel room booking operations.



Task 4 – Online Voting API Task:

Create a RESTful API for an online voting system.

Instructions:

- Endpoints required:
 - o GET /candidates → List candidates
 - o POST /vote → Cast a vote
 - o GET /results → View voting results
- Ensure one vote per user using in-memory validation.

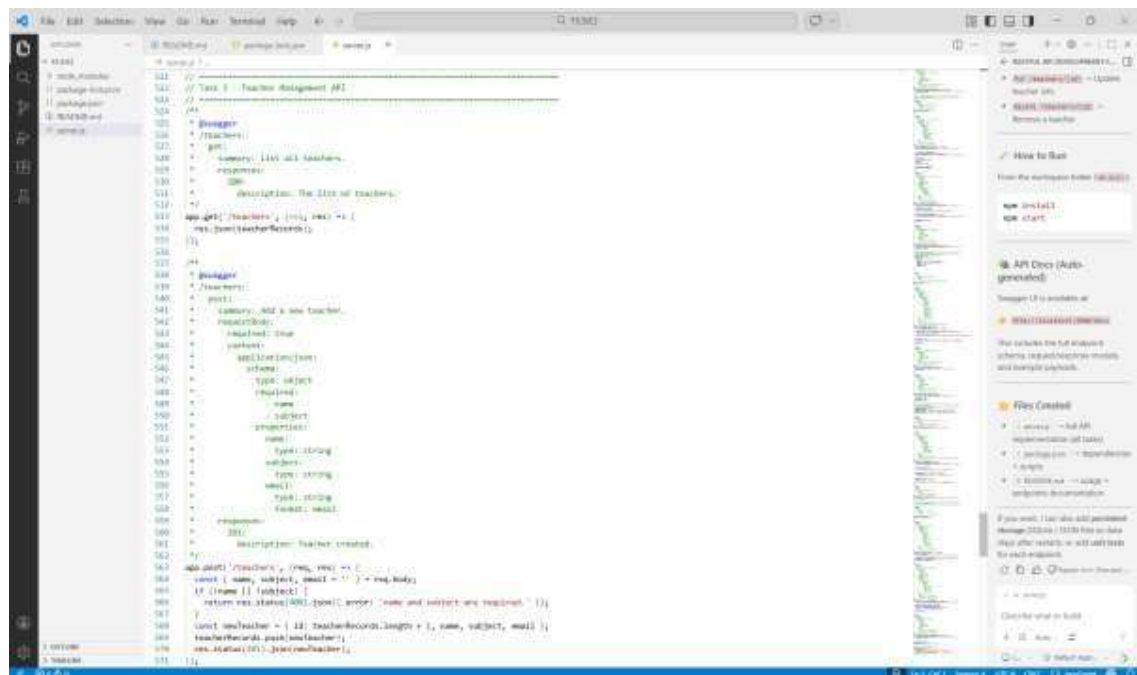
Expected Output:

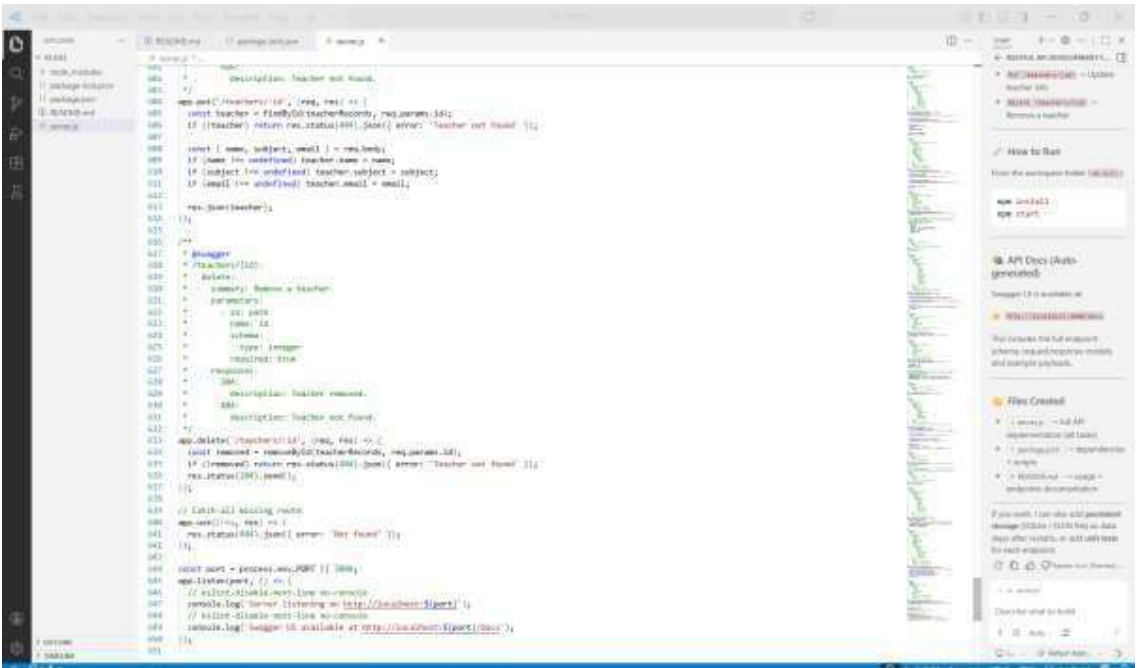
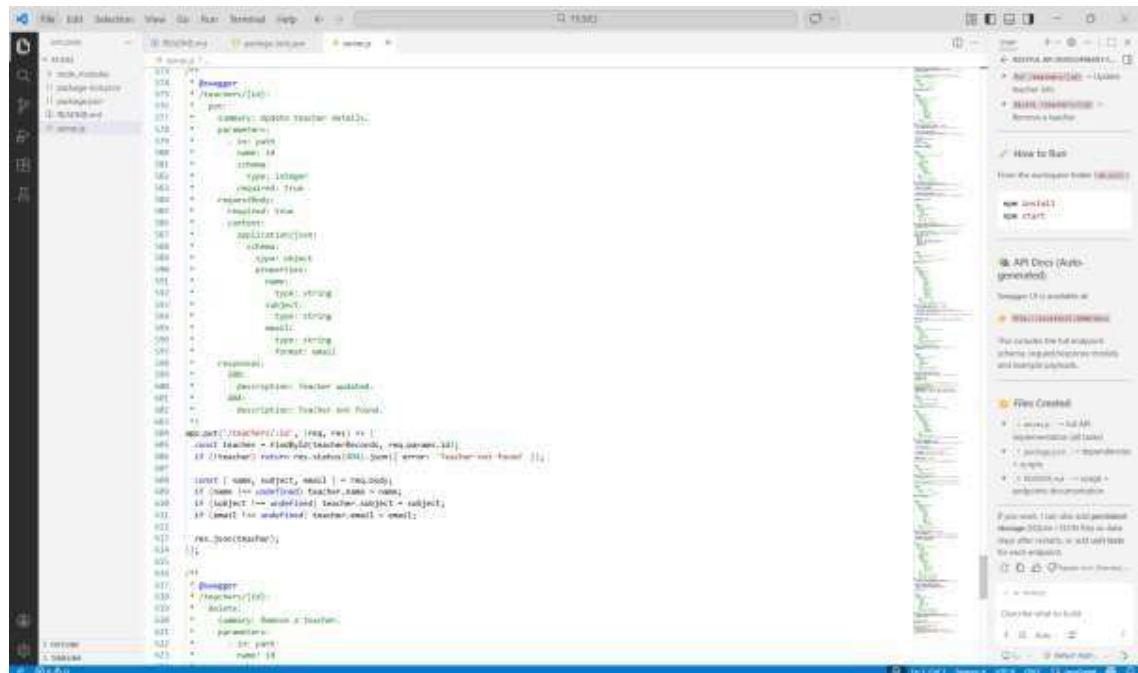
- API simulating a secure online voting process.

o GET /teachers → List all teachers o POST /teachers
 → Add a new teacher o PUT /teachers/{id} → Update
 teacher details o DELETE /teachers/{id} → Remove a
 teacher • Ensure proper error handling and JSON
 responses.

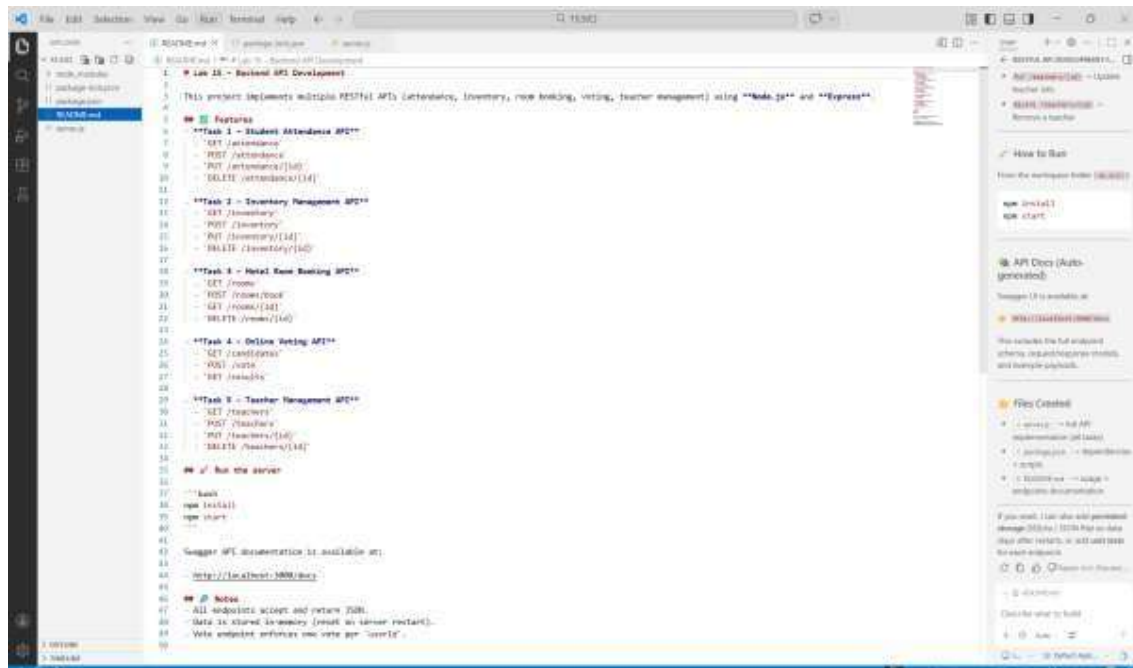
Expected Output:

- Functional API for managing teacher information.





Backend API Development

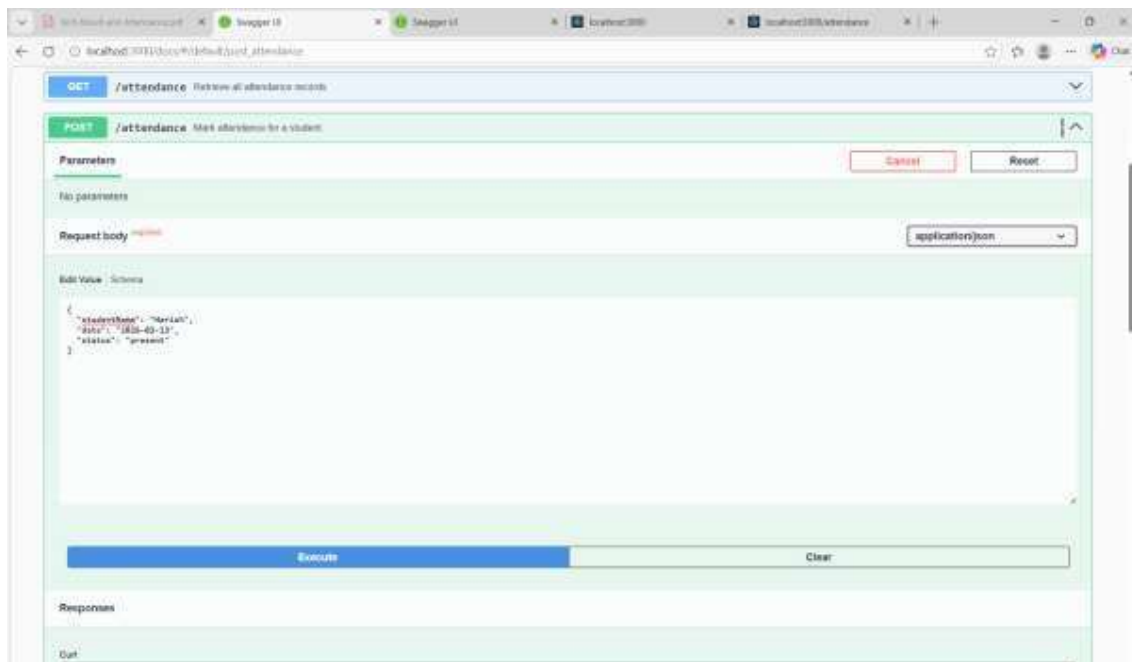
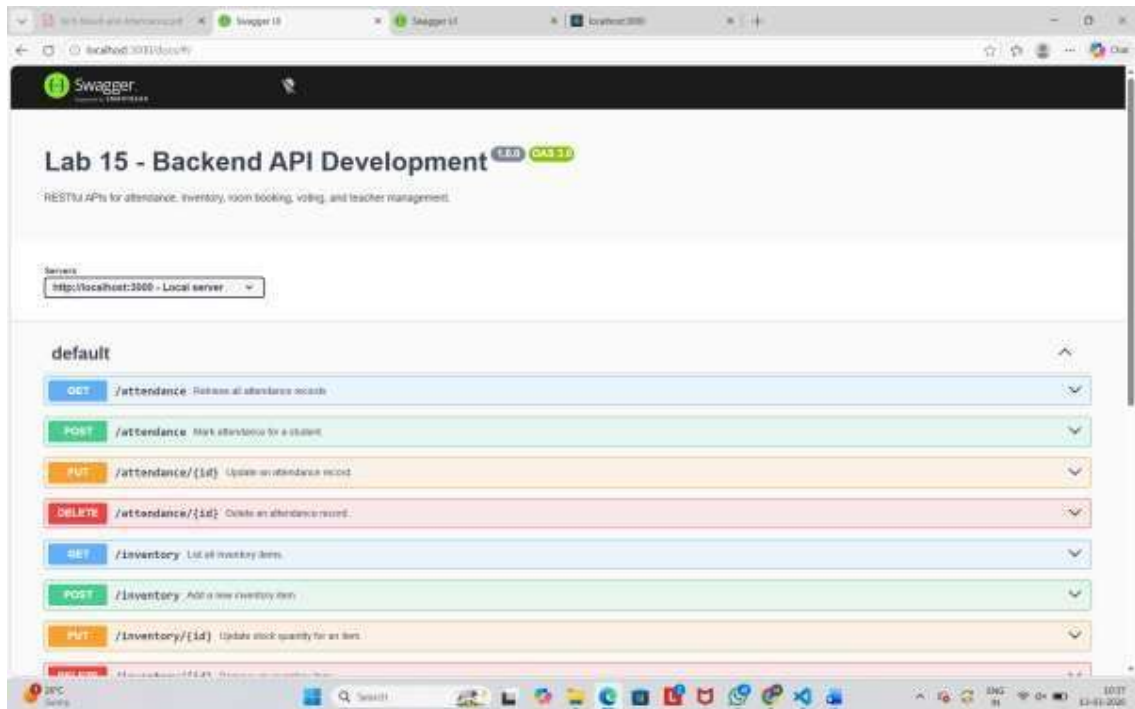


Install Project & Run the Project



Server listening on <http://localhost:3000>

Swagger UI available at <http://localhost:3000/docs> <http://localhost:3000/docs> :



Swagger UI interface showing a REST client request and response for the endpoint `/attendance/{id}`.

Request:

```
curl -X 'POST' \
  http://localhost:3000/attendance/ \
  -H 'Accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "studentName": "Harish",
    "date": "2025-03-13",
    "status": "present"
  }'
```

Response:

201

Response body:

```
{
  "id": 1,
  "studentName": "Harish",
  "date": "2025-03-13",
  "status": "present"
}
```

Response headers:

```
connection: keep-alive
content-length: 70
content-type: application/json; charset=utf-8
date: Fri, 13 Mar 2025 05:02:40 GMT
etag: W/"36-ua7c218802af9eb20a4b47b"
keep-alive: timeout=5
powered-by: Express
```

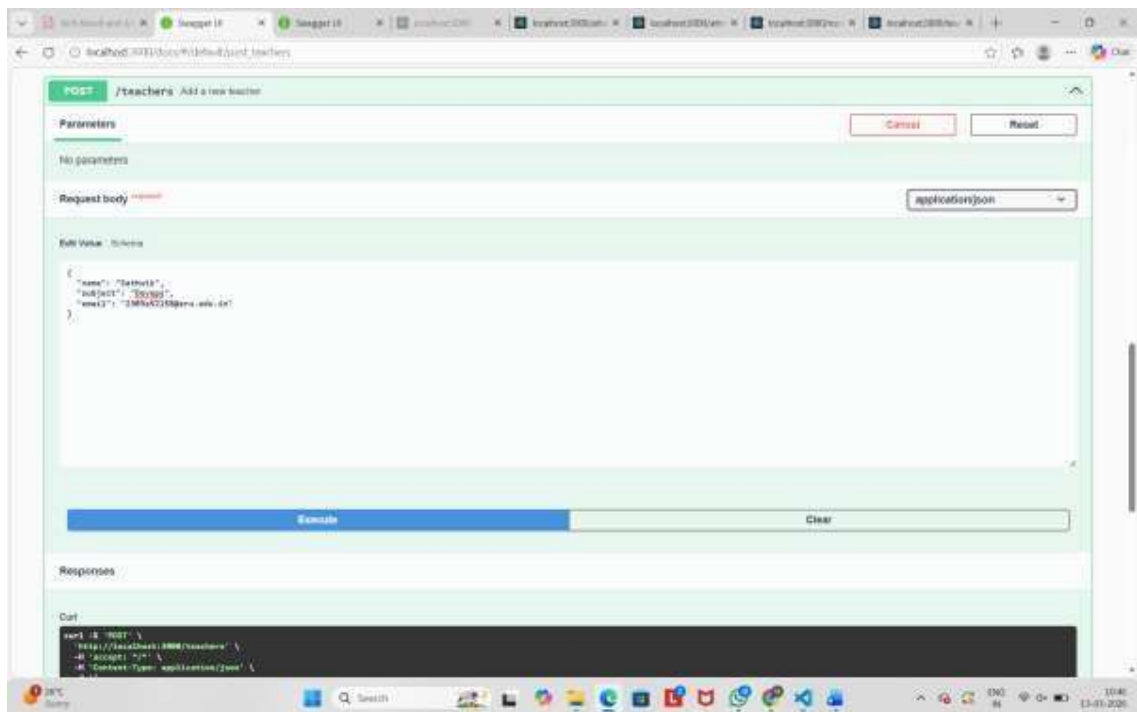
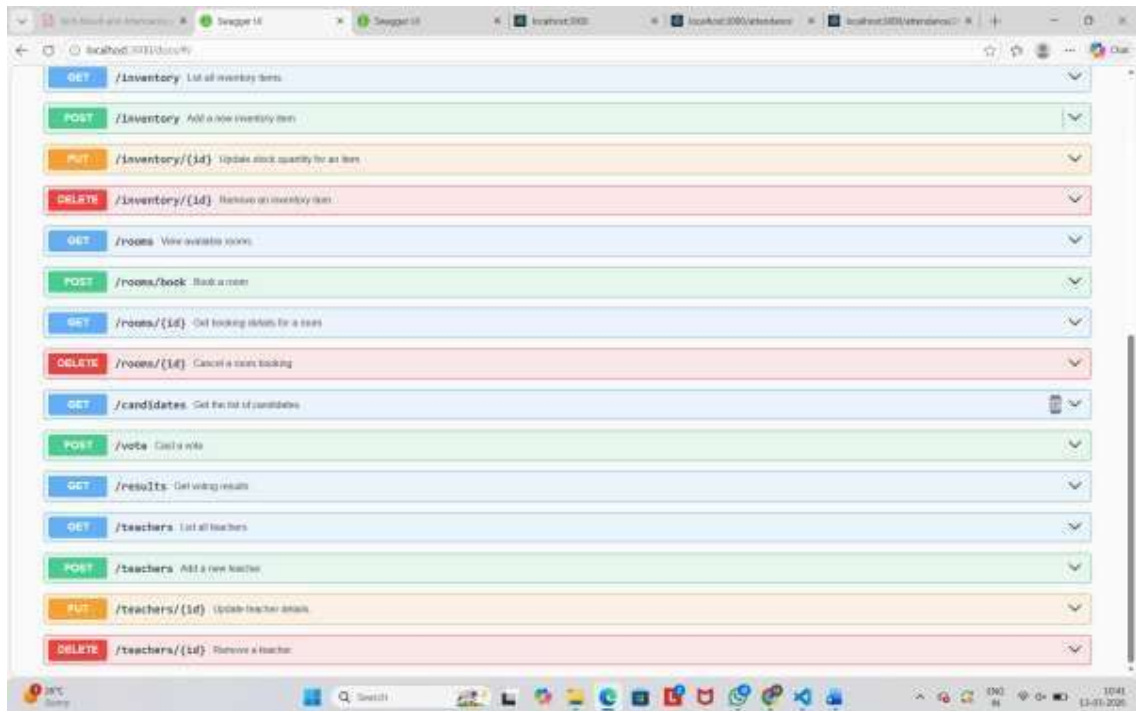
Summary Table:

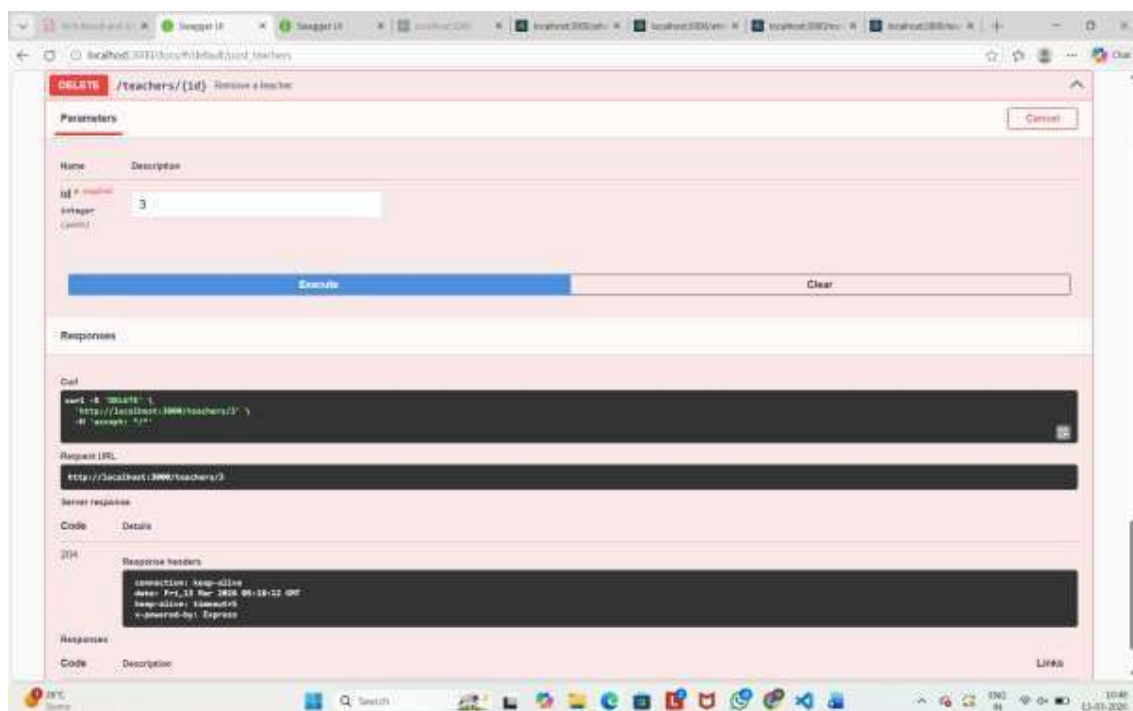
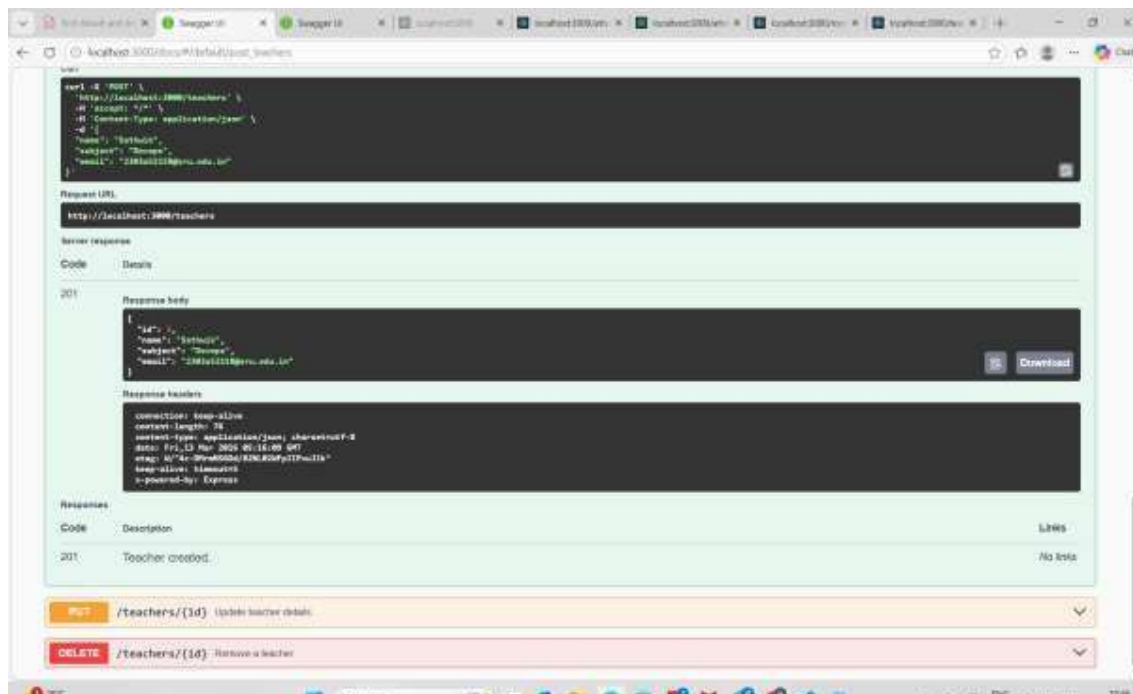
Code	Description	Links
201	Attendance recorded!	No links

POST /attendance/{id} Update an attendance record

Swagger UI interface showing the JSON response body for the endpoint `/attendance/{id}`.

```
{
  "id": 1,
  "studentName": "Harish",
  "date": "2025-03-13",
  "status": "present"
},
{
  "id": 2,
  "studentName": "Harish",
  "date": "2025-03-13",
  "status": "present"
},
{
  "id": 3,
  "studentName": "Harish",
  "date": "2025-03-13",
  "status": "present"
}
```





GITHUB LINK : <https://github.com/2303A52387/AI-ASSISTANT-CODING/blob/main/15.zip>